

Icarus Verilog for Verilog Simulation

PRIYA RAVINDRAN

With the VLSI industry booming, the need for electronic design automation (EDA) tools is increasing by the day. While proprietary tools serve their purpose efficiently, these cost you a bomb. This month's EFY Plus DVD introduces you to the latest version of a popular open source tool for simulating your Verilog code, Icarus Verilog.

Icarus Verilog is a software solution compatible with Institute of Electrical and Electronics Engineers (IEEE) 1364 standard for Verilog, IEEE 1800 for SystemVerilog and IEEE 1076 for very-high-speed integrated circuit hardware description language (VHDL). Based on GNU GPL copyleft licence, the tool is maintained by Stephen Williams and can perform batch simulations as well. This is a work-in-progress tool that changes with changing Verilog and supports Linux mainly, but also works well on other operating systems.

Features to make your job easy

When you sit down to write your code, there are portions like reading a register value directly or calculating delay that might be easier with C or not possible with Verilog. In such situations, you can use Verilog procedural interface (VPI) module, which is basically a programming language interface (PLI). This module allows you to interface your Verilog simulation with a code in C. It even has a Cadence PLI module that allows Icarus Verilog to run a file that was compiled to be loaded into VerilogXL or NC-Verilog.

To make you enjoy your simulation experience, Icarus Verilog provides an interactive module (IVI) supporting a variety of simulators. Vtracer, a Verilog testbench de-

Are you a beginner? Here is what you need

Hardware design generally follows a standard pattern; design entry, simulation, synthesis, layout, verification and fabrication. Icarus Verilog is a tool that aids the front end of your design, that is, till the synthesis stage.

Design entry. Describe your design in words or using schematics. The word description is generally in Verilog, VHDL or other such HDLs.

Simulation. Check the functionality of your design. Simulate how your design would work under different conditions by assigning values to the inputs in your design, via a test bench. Observe the outputs as display or waveform.

Synthesis. Convert your Verilog code into a machine-understandable format called netlist, which is simply a list of all wires and connections in your design. The netlist is what is carried forward from the front end into the layout stage, where the design is converted into a pictorial format, which is then printed. This stage is technology-specific and may also include analysis for timing and power factors.

How do you go about working with Icarus Verilog

1. Create your Verilog program in a text file with .v (ver or .vlg are also used) extension
2. Compile the code with the command `iverilog` (Make sure to add -o <file> to indicate where you want the compiled code to be stored)
3. Execute the compiled code using the command `vvp`

Here are a few links to help you get into the Icarus Verilog environment:

List of attributes: iverilog.wikia.com/wiki/Iverilog_Attributes

Basic commands: linux.die.net/man/1/iverilog

Icarus language: iverilog.wikia.com/wiki/Command_File_Format

Git repository (GitHub): github.com/steveicarus/iverilog

Documentation: iverilog.wikia.com/wiki/Main_Page

Website: iverilog.icarus.com

veloper aid, supports Verilog-Perl co-simulation. There is also an application program interface, an API module, which performs an action similar to reverse engineering.

You might have a Verilog code that is simulated and synthesised but does not behave exactly like the input code. In that case you can simply feed the elaborated design to this module, which will then write a code that Icarus Verilog can simulate.

Generate in the format you need.

It is now easy to get an output in the

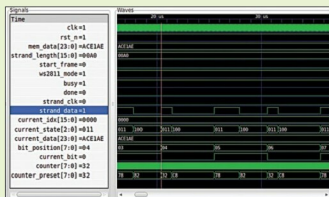


Fig. 1: A simulated waveform snapshot (Image courtesy: compostbinplans.org/icarus-verilog-simulator.htm)

electronic design interchange format (EDIF) using the EDIF loadable target module. There is also a field programmable gate arrays (FPGAs) target to Icarus Verilog that performs most of these functions and an extendable code generator to attach new output formatters to the compiler.